

Healthcare Medical Project

Modelo de Datos



David Mourenza Fantáns · Enrique Jiménez Martínez · Gael Díaz López · Paula Domínguez Reig

Máster FP Dual — Inteligencia Artificial y Big Data · Curso 2025-2026
NTT Data, A Coruña

Índice:	1
1. Descripción del dataset	3
2. Tablas del Dataset.....	4
2.1 patients — Dimensión Principal	4
2.2 diagnoses — Hechos de visitas	5
2.3 medications — Hechos de medicación	5
2.4 lab_results — Hechos de laboratorio	5
2.5 outcomes — Hechos de hospitalizaciones	6
3. Modelo de datos	7
3.1 Cardinalidad 1:N.....	7
4. Ingesta desde la Fuente de Datos	8
5. Carga y Validación de Datos	9
6. Limpieza de datos	11

1. Descripción del dataset

Visión general del modelo de datos clínico con 100.000 pacientes y 5 tablas relacionadas

La fuente de datos utilizada corresponde al dataset *Patient Records: 100K Patients, 15 Conditions*, publicado por Sergio Nefedov en [Kaggle](https://www.kaggle.com/datasets/sergionefedov/patient-records-100k-patients-15-conditions). El conjunto de datos contiene información de 11.001 pacientes distribuida en cinco tablas en formato CSV relacionadas entre sí mediante el identificador único del paciente.

<https://www.kaggle.com/datasets/sergionefedov/patient-records-100k-patients-15-conditions>

La tabla *patients* actúa como tabla principal y contiene una fila por paciente, representando sus características demográficas, signos vitales, hábitos de vida y comorbilidades conocidas.

Las cuatro tablas restantes están vinculadas a través de la clave *patient_id*, y registran los eventos clínicos asociados a cada paciente a lo largo del tiempo: visitas médicas (diagnoses), prescripciones farmacológicas (medications), resultados de laboratorio (*lab_results*) y episodios hospitalarios (*outcomes*).

Cada columna representa una característica específica del paciente o del evento clínico. Las variables numéricas continuas como *bmi*, *heart_rate* o los resultados de laboratorio proporcionan información cuantitativa sobre el estado de salud del paciente, mientras que las variables categóricas como *smoking_status*, *insurance_type* o *visit_type* permiten segmentar y contextualizar los registros.

La variable objetivo para los modelos de aprendizaje automático se encuentra en la tabla *outcomes* y es *readmitted_30d* (reingreso en 30 días).

El resto de las columnas de las cinco tablas actuarán como características de entrada para el modelo predictivo.

2. Tablas del Dataset

Descripción detallada de las cinco tablas: una dimensión principal y cuatro tablas de hechos

2.1 patients — Dimensión Principal

100.000 filas · Información demográfica, signos vitales, hábitos de vida y comorbilidades del paciente

Columna	Tipo	Descripción
<i>patient_id</i>	String	PK - Identificador único del paciente.
<i>age</i>	Integer	Edad del paciente, 18 y 95 años (En el dataset actual)
<i>sex</i>	String	Sexo biológico: M / F.
<i>bmi</i>	Double	Índice de Masa Corporal (IMC), valores entre 16.0 y 55.0.
<i>systolic_bp</i> / <i>diastolic_bp</i>	Integer	Presión arterial sistólica y diastólica expresadas en mmHg.
<i>heart_rate</i>	Integer	Frecuencia cardíaca en pulsaciones por minuto (bpm).
<i>temperature_f</i>	Double	Temperatura corporal en grados Fahrenheit (F).
<i>smoking_status</i>	String	Historial de tabaquismo: never / former / current.
<i>alcohol_use</i>	String	Nivel de consumo de alcohol: none, light, moderate o heavy.

Columna	Tipo	Descripción
exercise_level	String	Nivel de actividad física: sedentary, light, moderate o vigorous.
insurance_type	String	Tipo de cobertura: commercial, medicare, medicaid, uninsured o tricare.
charlson_index	Integer	Índice de Charlson de comorbilidad, valores desde 0 hasta 8+.
dx_* (x15 columnas)	Integer	0/1 Flags binarios de condiciones crónicas: hypertension, type2_diabetes, etc.

2.2 diagnoses — Hechos de visitas

~274.592 filas · Registro de diagnósticos clínicos por visita médica

Columna	Tipo	Descripción
patient_id	String	FK - Referencia al paciente en la tabla patients.
visit_date	Date	Fecha en que se realizó la visita médica.
visit_type	String	Modalidad de la visita: outpatient, inpatient, emergency o telehealth.
primary_diagnosis	String	Nombre textual del diagnóstico principal registrado.
primary_icd10	String	Código ICD-10 correspondiente al diagnóstico principal.
secondary_diagnoses	String	Diagnósticos secundarios, separados por delimitador.
secondary_icd10s	String	Códigos ICD-10 de los diagnósticos secundarios.
provider_specialty	String	Especialidad médica del profesional que atendió la visita.

2.3 medications — Hechos de medicación

~364.174 filas · Prescripciones farmacológicas y adherencia al tratamiento

Columna	Tipo	Descripción
patient_id	String	FK - Referencia al paciente en la tabla patients.
medication	String	Nombre del medicamento prescrito.
dose	Double	Dosis administrada por toma.
unit	String	Unidad de la dosis: mg, mcg, units, etc.
frequency	String	Frecuencia de administración: daily, twice daily, as needed...
indication	String	Motivo clínico de la prescripción (hipertensión, diabetes, etc.).
start_date	Date	Fecha de inicio del tratamiento.
duration_days	Integer	Duración total de la prescripción en días.
is_generic	Integer	0/1 Indica si el medicamento es un genérico (1) o de marca (0).
adherence_pct	Double	Porcentaje de adherencia al tratamiento, de 0 a 100.

2.4 lab_results — Hechos de laboratorio

~2.827.722 filas · Resultados numéricos de pruebas de laboratorio clínico

Columna	Tipo	Descripción
patient_id	String	FK - Referencia al paciente en la tabla patients.
test_date	Date	Fecha en que se realizó el análisis.
test_name	String	Nombre de la prueba: WBC, HbA1c, creatinine, etc.
value	Double	Valor numérico del resultado obtenido.
unit	String	Unidad de medida del resultado (g/dL, mmol/L, etc.).
reference_low / reference_high	Double	Límites inferior y superior del rango de referencia normal.
flag	String	Indicador de resultado: normal, high o low.
is_abnormal	Integer	0/1 Indica si el resultado está fuera del rango normal (1) o no (0).
delta_from_normal	Double	Desviación numérica del valor respecto al rango de referencia.

2.5 outcomes — Hechos de hospitalizaciones

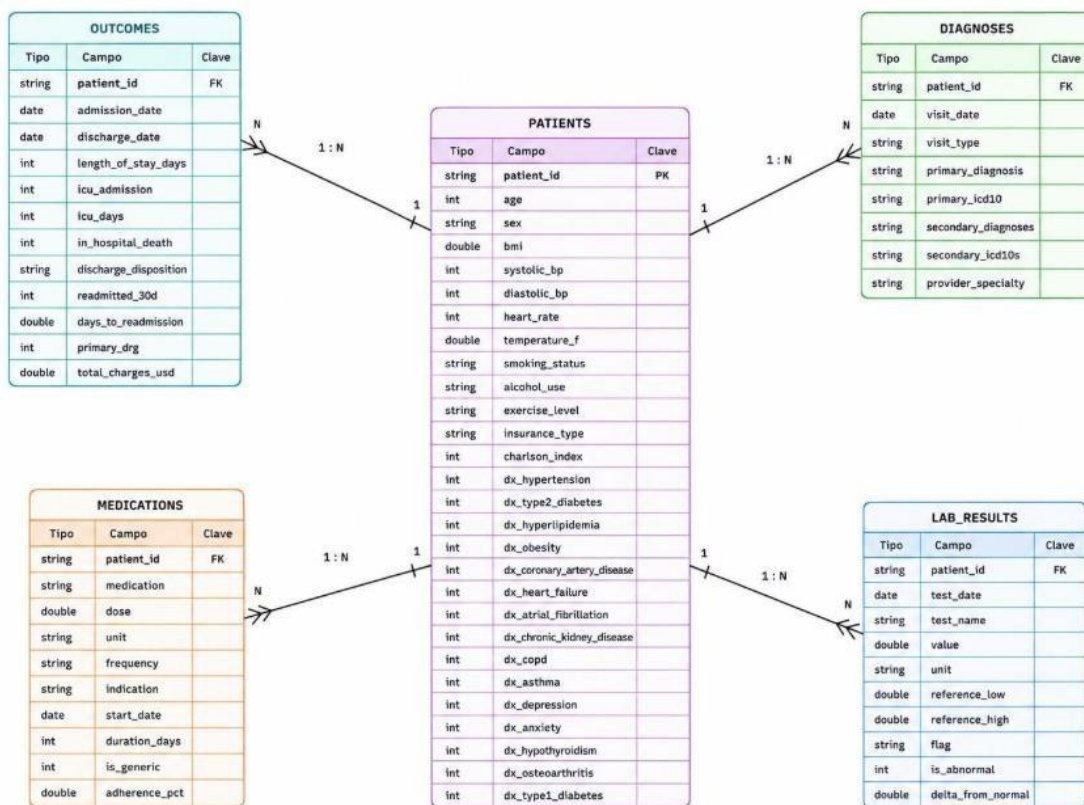
~11.001 filas · Episodios hospitalarios y resultados clínicos. Contiene la variable objetivo **readmitted_30d**

Columna	Tipo	Descripción
patient_id	String	FK - Referencia al paciente en la tabla patients.
admission_date	Date	Fecha de ingreso hospitalario.
discharge_date	Date	Fecha de alta hospitalaria.
length_of_stay_days	Integer	Duración total de la estancia hospitalaria en días.
icu_admission	Integer	0/1 Indica si el paciente requirió ingreso en UCI (1) o no (0).
icu_days	Integer	Número de días de estancia en la Unidad de Cuidados Intensivos.
in_hospital_death	Integer	0/1 Indica si el paciente falleció durante el ingreso (1) o no (0).
discharge_disposition	String	Destino al alta: home, rehab, hospice, expired...
readmitted_30d	Integer	0/1 Indica si el paciente fue reingresado en los 30 días siguientes al alta.
days_to_readmission	Double	Número de días transcurridos hasta el reingreso.
primary_drg	Integer	Código DRG (Diagnosis Related Group) del episodio principal.
total_charges_usd	Double	Coste total del episodio hospitalario en dólares (USD).

La variable objetivo para los modelos de aprendizaje automático será **readmitted_30d**. El resto de las columnas de las cinco tablas actuarán como características de entrada del modelo.

3. Modelo de datos

Esquema Estrella (Star Schema) centrado en el paciente



El modelo de datos sigue un Esquema Estrella (Star Schema) centrado en el paciente. La tabla PATIENTS ocupa el centro del esquema y actúa como tabla de dimensión principal, mientras que las cuatro tablas de hechos — DIAGNOSES, MEDICATIONS, LAB_RESULTS y OUTCOMES — se sitúan alrededor, conectadas mediante la clave patient_id.

3.1 Cardinalidad 1:N

Un único paciente puede tener múltiples registros en cada tabla de hechos

Todas las relaciones del esquema tienen una cardinalidad de uno a muchos (1:N), lo que significa que un único paciente puede tener asociados múltiples registros en cada tabla de hechos:

- Un paciente puede tener múltiples visitas médicas registradas en **diagnoses**.
- Un paciente puede tener múltiples prescripciones a lo largo del tiempo en **medications**.
- Un paciente puede tener múltiples resultados de laboratorio en **lab_results**.
- Un paciente puede tener múltiples episodios de hospitalización recogidos en **outcomes**.

La clave primaria patient_id en la tabla PATIENTS (PK) se convierte en clave foránea (FK) en cada una de las cuatro tablas de hechos, garantizando la integridad referencial del modelo.

4. Ingesta desde la Fuente de Datos

Descarga automatizada desde Kaggle con gestión segura de credenciales en Azure Key Vault

Inicio > Almacenes de claves > kv-medicalproject

kv-medicalproject | Secretos

Almacén de claves

☆ ...

×

Buscar

+

Generar o importar

↺

Actualizar

↶

Restaurar copia de seguridad

🔗

Administrar secretos eliminados

🔗

Ver código de ejemplo

Información general

Registro de actividad

Control de acceso (IAM)

Etiquetas

Diagnosticar y solucionar problemas

Directivas de acceso

Visualizador de recursos

Eventos

Objetos

Claves

Secretos

☆

Certificados

Configuración

Configuración de acceso

Nombre	Tipo	Estado	Fecha de expiración
KaggleApiToken		✓ Habilitada	
KaggleUser		✓ Habilitada	
OpenCodeGoKey		✓ Habilitada	
TelegramChatId		✓ Habilitada	
TelegramToken		✓ Habilitada	

Se consumen los datos directamente desde Kaggle para inyectarlos en la capa de almacenamiento origen (**raw**).

```
# -- Variables de conexión --
# El Key Vault almacena las credenciales de Kaggle de forma segura
NOMBRE_BOVEDA = "kv-medicalproject"
NOMBRE_PUENTE = "claveskaggle" # Nombre del servicio vinculado en Synapse
URL_DATASET = "https://www.kaggle.com/api/v1/datasets/download/sergionefedov/patient-records-100k-patients-15-conditions"

try:
    # Obtenemos las credenciales desde Azure Key Vault
    logger.info("Obteniendo credenciales de Kaggle desde Key Vault...")
    kaggle_user = mssparkutils.credentials.getSecret(NOMBRE_BOVEDA, "KaggleUser", NOMBRE_PUENTE)
    kaggle_key = mssparkutils.credentials.getSecret(NOMBRE_BOVEDA, "KaggleApiToken", NOMBRE_PUENTE)
    logger.info("Credenciales obtenidas correctamente.")
except Exception as e:
    logger.error(f"Error al obtener credenciales de Key Vault: {e}")
    logger.error("Verifica que el Key Vault y el servicio vinculado estén configurados.")
    raise
```

Para ello, se realiza una descarga automatizada mediante peticiones HTTPS autenticadas, procesando el archivo comprimido **completamente en memoria** para optimizar el rendimiento y evitar archivos temporales.

El proceso garantiza la **idempotencia** mediante la sobrescritura explícita (`overwrite=True`) en el destino. Asimismo, se integra **Azure Key Vault** (`kv-medicalproject`) para la recuperación segura de las API keys (`KaggleUser` y `KaggleApiToken`), mitigando el riesgo de exponer credenciales en el código fuente.

```
logger.info("Descargando dataset desde Kaggle...")

# Descargamos el ZIP completo en memoria (evita escribir archivos temporales)
respuesta = requests.get(URL_DATASET, auth=(kaggle_user, kaggle_key))
respuesta.raise_for_status()

logger.info(f"ZIP descargado correctamente ({len(respuesta.content):,} bytes).")
```

```
# -- Extraemos el ZIP en memoria y subimos cada CSV a Azure --
RUTA_RAW = f"abfss://{CONTAINER_RAW}@{STORAGE_ACCOUNT}.dfs.core.windows.net"
archivo_zip = zipfile.ZipFile(io.BytesIO(respuesta.content))
archivos_en_zip = archivo_zip.namelist()

logger.info(f"Archivos encontrados en el ZIP: {archivos_en_zip}")

archivos_subidos = 0
for nombre_archivo in archivos_en_zip:
    # Ignoramos archivos del sistema (MACOSX, etc.)
    if nombre_archivo.startswith('__') or nombre_archivo.startswith('.'):
        continue

    logger.info(f"Subiendo {nombre_archivo} a {RUTA_RAW}/...")
    contenido_csv = archivo_zip.read(nombre_archivo).decode('utf-8')
    ruta_destino = f"{RUTA_RAW}/{nombre_archivo}"
    mssparkutils.fs.put(ruta_destino, contenido_csv, True)
    archivos_subidos += 1

logger.info(f"Subida completada: {archivos_subidos} archivos enviados a {RUTA_RAW}/")
```

Para realizar la descarga de los datos e inyectarlos en la carpeta raw se realizaron estos 2 anteriores bloques de código:

5. Carga y Validación de Datos

Procesamiento inicial de registros y control de tipos de datos

```
archivos = {
    "patients": ("patients.csv", esquema_patients),
    "outcomes": ("outcomes.csv", esquema_outcomes),
    "medications": ("medications.csv", esquema_medications),
    "lab_results": ("lab_results.csv", esquema_lab_results),
    "diagnoses": ("diagnoses.csv", esquema_diagnoses),
}

dfs = {}
errores_carga = []

for nombre, (archivo, esquema) in archivos.items():
    try:
        ruta = f"{base_path}/{archivo}"
        dfs[nombre] = spark.read.csv(ruta, header=True, schema=esquema, enforceSchema=True)
        total = dfs[nombre].count()
        logger.info(f"Cargado '{archivo}' - {total:,} registros.")
    except Exception as e:
        errores_carga.append((archivo, str(e)))
        logger.error(f"ERROR al cargar '{archivo}': {e}")

if errores_carga:
    logger.error("Se encontraron errores en la carga. Revise los archivos.")
    raise RuntimeError(f"Falló la carga de {len(errores_carga)} archivo(s).")

logger.info(f"Carga completada - {len(dfs)} tablas en memoria.")

esquemas = {
    "patients": esquema_patients,
    "outcomes": esquema_outcomes,
    "medications": esquema_medications,
    "lab_results": esquema_lab_results,
    "diagnoses": esquema_diagnoses,
}

resultados_tipado = []

for nombre, df in dfs.items():
    esquema_esperado = esquemas[nombre]
    tipos_esperados = {f.name: f.dataType.simpleString() for f in esquema_esperado.fields}
    tipos_reales = dict(df.dtypes)

    errores = []
    for col, tipo_esp in tipos_esperados.items():
        if col not in tipos_reales:
            errores.append(f"Falta columna '{col}'")
        elif tipos_reales[col] != tipo_esp:
            errores.append(f"'{col}': esperado={tipo_esp}, real={tipos_reales[col]}")
    for col in tipos_reales:
        if col not in tipos_esperados:
            errores.append(f"Columna extra: '{col}'")

    estado = "OK" if not errores else f"{len(errores)} problema(s)"
    detalle = "; ".join(errores) if errores else "Todos los tipos coinciden"
    resultados_tipado.append({"Tabla": nombre, "Estado": estado, "Detalle": detalle})
    logger.info(f" {nombre:<15} {estado}")

df_tipado = pd.DataFrame(resultados_tipado)
display(df_tipado)
```

Se realiza la carga de los distintos archivos CSV (patients, outcomes, medications, lab_results y diagnoses) en DataFrames de PySpark mediante la función `spark.read.csv()`, permitiendo trabajar con la información clínica dentro del entorno de procesamiento. Posteriormente, los DataFrames se agrupan en un diccionario para facilitar su recorrido y análisis conjunto.

El código realiza las siguientes comprobaciones por cada tabla:

- **Volumetría:** Registra el total de filas cargadas en memoria mediante la función `.count()`.
- **Validación de estructura y tipado:** Compara las columnas y tipos de datos reales (`df.dtypes`) contra un esquema clínico predefinido y obligatorio (`enforceSchema=True`).
- **Detección de anomalías estructurales:** Identifica si faltan columnas obligatorias, si existen columnas extra no deseadas o si el tipo de dato real no coincide con el esperado.
- **Consolidación de alertas:** Centraliza los resultados en los logs del sistema y construye un DataFrame ejecutivo (`df_tipado`) que detalla el estado final ("OK" o "Problema(s)") de cada tabla analizada.

6. Limpieza de datos

Se realiza la estandarización de datos en PySpark

```
# Estandarizamos los datos: ajustamos formatos de texto, convertimos unidades (F->C), normalizamos tipos y aplicamos reglas de negocio
logger.info("Aplicando estandarización Estructural, Semantica y Temporal...")

# =====
# 1. PATIENTS (Maestro)
# =====
df_patients_clean = df_patients \
    .withColumn("patient_id", trim(upper(col("patient_id")))) \
    .withColumn("sex", trim(upper(col("sex")))) \
    .withColumn("smoking_status", trim(lower(col("smoking_status")))) \
    .withColumn("insurance_type", trim(lower(col("insurance_type")))) \
    .withColumn("age", col("age").cast("integer")) \
    .withColumn("bmi", col("bmi").cast("double"))

# Conversion a Celsius (redondeo a 1 decimal)
df_patients_clean = df_patients_clean \
    .withColumn("temperature_c", round((col("temperature_f") - 32) * 5.0 / 9.0, 1)) \
    .drop("temperature_f")

# Convertir dinamicamente todos los "dx_..." a Booleanos
columnas_dx = [c for c in df_patients_clean.columns if c.startswith("dx_")]
for c in columnas_dx:
    df_patients_clean = df_patients_clean.withColumn(c, col(c).cast("boolean"))

logger.info("    -> Tabla Patients limpiada (Fahrenheit a Celsius y Diagnosticos a Booleanos).")

# =====
# 2. OUTCOMES (Eventos)
# =====
df_outcomes_clean = df_outcomes \
    .withColumn("patient_id", trim(upper(col("patient_id")))) \
    .withColumn("discharge_disposition", trim(lower(col("discharge_disposition")))) \
    .withColumn("admission_date", to_date(col("admission_date"), "yyyy-MM-dd")) \
    .withColumn("discharge_date", to_date(col("discharge_date"), "yyyy-MM-dd")) \
    .withColumn("total_charges_usd", col("total_charges_usd").cast("double")) \
    .withColumn("icu_admission", col("icu_admission").cast("boolean")) \
    .withColumn("in_hospital_death", col("in_hospital_death").cast("boolean")) \
    .withColumn("readmitted_30d", col("readmitted_30d").cast("boolean"))

# Regla Temporal: Filtro de coherencia de fechas (se mantienen como DateType)
df_outcomes_clean = df_outcomes_clean.filter(col("discharge_date") >= col("admission_date"))

logger.info("    -> Tabla Outcomes limpiada (Filtro temporal aplicado, fechas como DateType).")
```

```
# =====
# 3, 4 y 5. MEDICATIONS, LAB_RESULTS y DIAGNOSES
# =====
df_medications_clean = df_medications \
    .withColumn("patient_id", trim(upper(col("patient_id")))) \
    .withColumn("medication", trim(lower(col("medication")))) \
    .withColumn("unit", trim(lower(col("unit")))) \
    .withColumn("dose", col("dose").cast("double")) \
    .withColumn("start_date", to_date(col("start_date"), "yyyy-MM-dd")) \
    .withColumn("is_generic", col("is_generic").cast("boolean"))
logger.info("    -> Tabla Medications limpiada.")

df_lab_results_clean = df_lab_results \
    .withColumn("patient_id", trim(upper(col("patient_id")))) \
    .withColumn("test_name", trim(lower(col("test_name")))) \
    .withColumn("unit", trim(lower(col("unit")))) \
    .withColumn("value", col("value").cast("double")) \
    .withColumn("test_date", to_date(col("test_date"), "yyyy-MM-dd")) \
    .withColumn("is_abnormal", col("is_abnormal").cast("boolean"))
logger.info("    -> Tabla Lab Results limpiada.")

df_diagnoses_clean = df_diagnoses \
    .withColumn("patient_id", trim(upper(col("patient_id")))) \
    .withColumn("primary_diagnosis", trim(lower(col("primary_diagnosis")))) \
    .withColumn("primary_icd10", trim(upper(col("primary_icd10")))) \
    .withColumn("visit_date", to_date(col("visit_date"), "yyyy-MM-dd"))
logger.info("    -> Tabla Diagnoses limpiada.")
# Subida parcial de log tras esta seccion
_subir_log("silver/silver_integridad_referencial")
```

Como se aprecia en las capturas, el pipeline ejecuta una estandarización de los DataFrames de PySpark aplicando transformaciones internas para asegurar su consistencia:

- Normalización de texto: Elimina espacios residuales (trim) y unifica formatos usando mayúsculas (upper) para identificadores formales (como patient_id) y minúsculas (lower) para variables categóricas.

- Casteo y cálculos: Ajusta estructuralmente los tipos de datos (Integer, Double, Date, Boolean). Además, aplica conversiones aritméticas (Fahrenheit a Celsius) y convierte dinámicamente las columnas de diagnóstico (dx_...) a booleanos.
- Reglas de negocio y auditoría: Filtra los registros para mantener la coherencia temporal (asegurando que la fecha de alta sea igual o posterior al ingreso) y registra cada paso completado en el sistema de logs.

Archivo CSV	Total Filas	Total Nulos	Columnas Afectadas
patients.csv	100.000	0	Limpio
outcomes.csv	11.001	9.358	days_to_readmission (9.358)
medications.csv	364.174	0	Limpio
lab_results.csv	2.827.722	0	Limpio
diagnoses.csv	274.592	228.274	secondary_diagnoses (114.137), secondary_icd10s (114.137)

En la tabla anterior se muestran los resultados de la auditoría sobre los cinco archivos CSV del dataset.

La variable objetivo para los modelos de aprendizaje automático es **readmitted_30d**. El resto de las columnas de las cinco tablas actuarán como características de entrada del modelo.

7. Pipelines

Contamos con 2 pipelines, una que va ejecutando el código enviando las alertas de cada paso a Telegram y otra que captura cualquier error en el pipeline principal.

